Vector Search Is Not Enough:

The Case for Hybrid Retrieval in Mission-Critical Legal Document Applications

Working Paper — Draft for Review | March 4, 2026
Domain: Insurance Law · Corporate Policy · Product Policy · Regulatory Compliance

Abstract

Applications that generate proposals, recommendations, or decisions against regulatory, insurance, or corporate policy corpora operate in a domain where retrieval errors carry legal consequences. This paper examines why vector similarity search alone is structurally insufficient for such applications and recommends a hybrid retrieval architecture — combining lexical search, semantic vector retrieval, structured graph traversal, and cross-encoder reranking fused by Reciprocal Rank Fusion — as one effective approach to mission-critical legal document retrieval. It is important to note that this is not the only viable approach; the field offers a rich landscape of alternative techniques, several of which are catalogued in Appendix C. The primary purpose of this paper is to expose the structural weaknesses of vector-only retrieval and present one implementation that the author has found effective in practice. We further recommend that retrieval architecture be treated as downstream of knowledge governance: document versioning, lifecycle management at ingestion, and immutable decision provenance are architectural prerequisites that should be addressed before retrieval is designed.

1. Introduction

The application of large language models to legal, regulatory, and policy document work has accelerated rapidly. The dominant pattern — Retrieval Augmented Generation (RAG) — retrieves relevant document chunks from a corpus and presents them as context to an LLM, which generates a response. The retrieval step is treated, in most implementations, as a solved or near-solved problem: embed the corpus, embed the query, return the nearest vectors.

This paper examines that assumption in the specific context of mission-critical legal applications. We define mission-critical as any application where an AI-generated output — a proposal, recommendation, coverage determination, compliance assessment, or policy interpretation — has material legal consequences if wrong. Insurance underwriting decisions, regulatory compliance assessments, and corporate policy interpretations are canonical examples.

The goal is not to prescribe a single solution. The retrieval research community has produced a wide range of techniques that address the limitations of vector-only search in different ways and with different trade-offs (see Appendix C). Rather, this paper uses a working implementation as a concrete vehicle to expose where and why vector-only retrieval fails in legal document contexts, and to recommend one hybrid architecture the author has found effective in practice.

The failure mode of vector-only retrieval in this domain is not 'slightly less relevant results.' It is silent, confident retrieval of superseded, misscoped, or structurally inapplicable content — presented with similarity scores that provide no signal of the error. The downstream LLM generates coherent, well-structured outputs from this content. The outputs may be wrong in ways that are not detectable without expert review.

2. The Retrieval Problem in Legal Document Applications

2.1 Corpus Characteristics

Legal, regulatory, and policy corpora have structural properties that distinguish them from general-purpose knowledge bases:

- **Temporal validity.** Every document has an effective date. Insurance policies, regulatory filings, and corporate governance documents are superseded when amended. The content of a superseded document may be legally significant for historical decisions made under it, but is incorrect for current decisions. Both versions may coexist in the corpus simultaneously.
- **Jurisdictional scope.** A document applies within specific jurisdictional boundaries — federal, state, country, product line, business unit. A California surplus lines requirement does not apply to an admitted carrier in New York. Retrieval that ignores jurisdictional scope returns legally inapplicable content with no visible signal of the error.
- **Structural authority.** Legal documents have internal hierarchies where specific provisions govern general ones. An endorsement overrides the base policy. A definition in Section 1 governs every clause that uses the defined term. These structural relationships determine legal meaning in ways that flat chunk retrieval cannot represent.
- **Precision vocabulary.** Legal drafting uses terms of art with specific meanings that differ from everyday usage. 'Occurrence' versus 'claims-made' coverage triggers. 'Named insured' versus 'additional insured.' 'Surplus lines' versus 'admitted market.' The distinction between these terms is legally material. Retrieval that treats them as semantically similar fails at the threshold of the domain.

2.2 Proposal and Decision Requirements

Applications generating proposals or decisions against this corpus have requirements beyond relevant retrieval:

- **Version traceability.** A proposal generated today should be traceable to the specific document versions that were in effect at generation time. If those documents are later amended, the record of what was in force at decision time should be preserved independently and remain queryable.
- **Applicability confidence.** The application should be able to express appropriate uncertainty when retrieved evidence is insufficient, ambiguous, or conflicting. Returning a best-guess answer when the correct answer is 'the corpus does not support a determination' is a failure mode with potential legal consequences.
- **Auditability.** In regulated industries, the basis for AI-assisted decisions should be auditable by regulators, compliance officers, and in litigation. This requires not just logging what the system said, but what it knew — which document versions, which specific provisions, what confidence signals — at the time it said it.

3. Where Vector Search Falls Short

This section examines the structural characteristics of vector similarity search that make it insufficient as a standalone retrieval mechanism for legal document corpora. These are not implementation deficiencies — they are properties of the approach itself.

3.1 The Encoding Problem

Vector embeddings are a lossy compression of text meaning into a fixed-dimension floating-point space. The compression function is trained on large general-domain corpora. The properties it preserves are those that generalize across that training distribution: topical similarity, syntactic patterns, semantic proximity in everyday language use.

The properties it does not reliably preserve are those specific to legal documents: temporal validity, jurisdictional scope, structural authority, supersession relationships, and version identity. When a current and a superseded version of a regulatory requirement are both embedded, they land close together in vector space because they discuss the same regulatory topic in similar language. The embedding model has no representation of 'this version was replaced by that version on this date.' That information is not in its training distribution and cannot be recovered from the embedding at query time.

3.2 The Confidence Problem

Vector similarity scores are relative to the local density of the embedding space, not absolute measures of relevance or applicability. A corpus heavily weighted toward general liability policy language develops a dense cluster in that region of embedding space. Queries touching anything adjacent to liability return high similarity scores because the neighborhood is dense — regardless of whether the retrieved chunks apply to the specific jurisdiction or product type in question.

Standard score normalization — converting cosine distance to a 0-1 similarity value — compounds this problem. The normalized value looks authoritative but carries no information

about whether the retrieved chunk is currently in force, whether it applies to the queried jurisdiction, or whether the corpus contains a more applicable provision that scored lower due to distributional effects. A high similarity score for a superseded provision retrieved against the wrong jurisdiction has expressed false precision.

3.3 The Vocabulary Problem

Legal drafting is exact by design. The difference between 'occurrence-based' and 'claims-made' coverage is not a semantic nuance — it is a binary distinction that determines whether a claim is covered at all. Standard embedding models, trained on general text, may represent these terms as semantically proximate because they co-occur frequently in insurance contexts. A query about claims-made triggers may retrieve occurrence-based provisions with high similarity scores.

Lexical retrieval approaches — BM25 being one example — do not have this problem for exact terms. For precision legal vocabulary, exact token matching is a retrieval requirement, not a limitation to be engineered around.

3.4 The Structure Problem

Legal documents are not bags of chunks. They are hierarchically structured artifacts where the meaning of any provision depends on its relationship to other provisions. An endorsement that modifies the base policy requires knowing which policy it attaches to. A definition that scopes a coverage grant requires knowing which coverage grant it applies to.

Standard chunking and vector embedding destroys this structure. The retrieval system returns individual provisions without their structural context, and the LLM must infer structure from co-retrieved fragments — an inference that may be incorrect. Multiple retrieval techniques exist to address this at different layers of the stack (see Appendix C for a catalogue).

4. A Recommended Hybrid Architecture

The following describes one hybrid retrieval architecture the author has found effective for legal document retrieval in practice. It is presented as a concrete recommendation, not as the definitive or only viable solution. Alternative approaches with different trade-offs are catalogued in Appendix C, and practitioners should evaluate those options against their specific corpus characteristics, latency requirements, and operational constraints.

The architecture combines five components: BM25 lexical search, semantic vector retrieval with HyDE query expansion, knowledge graph traversal, Reciprocal Rank Fusion, and BGE cross-encoder reranking. Each component targets a specific failure mode identified in Section 3. The absence of any one component leaves a category of failure unaddressed — though practitioners may find that different combinations or alternative components serve the same purpose in their context.

4.1 Lexical Search: BM25 for Precision Vocabulary

BM25 (Okapi) provides lexical retrieval operating on exact token matches, weighted by term frequency and inverse document frequency with length normalization. For legal vocabulary, a lexical component is recommended as a complement to semantic retrieval — not a replacement for it.

BM25Okapi specifically addresses two limitations of raw TF-IDF scoring. Term frequency saturation prevents a provision that repeats a term many times from scoring proportionally higher than one that uses it precisely once. Length normalization prevents long provisions from dominating short ones on absolute term count rather than term density. Both corrections matter for legal text where section length varies enormously.

In the reference implementation, BM25 operates from an in-memory index built at startup and rebuilt after each ingestion event, delivering sub-millisecond query times with no network round-trip. Other lexical approaches — fielded BM25F, SPLADE sparse neural retrieval — may be preferable depending on infrastructure and corpus characteristics.

4.2 Vector Search with HyDE: Semantic Recall for Conceptual Queries

Vector similarity search handles semantic retrieval — finding provisions conceptually relevant to a query even when query and document use different vocabulary. This is essential for analysis use cases where the query does not match the exact terms of applicable provisions.

HyDE (Hypothetical Document Embeddings) improves recall for abstract queries by generating a hypothetical provision that would answer the query and embedding that instead of the raw query. The resulting vector lands closer to actual legal provisions in embedding space than a short query embedding would. This is one of several query expansion approaches; alternatives include query rewriting and RAG-Fusion.

4.3 Graph Traversal: Structural Relationships That Chunks Cannot Carry

A knowledge graph populated during ingestion — with entities, typed relationships, and structural connections between documents — enables traversal-based retrieval that flat search cannot replicate. At query time, graph traversal follows entity relationships (direct and related, 1-hop and 2-hop) and theme-based connections to surface documents through structural paths rather than similarity scores.

For regulatory corpora, the graph layer can carry versioning and supersession relationships, enabling temporal constraint enforcement through traversal rather than post-retrieval filtering. This is one structural approach; alternatives such as parent-document retrieval, hierarchical chunking, and RAPTOR-style summary indexing address related problems from different angles.

4.4 Reciprocal Rank Fusion: Signal-Agnostic Fusion

RRF combines ranked outputs of all retrieval signals without requiring score normalization across incompatible spaces. The formula score = Σ 1/(k + rank + 1) uses only ordinal position,

making it insensitive to magnitude differences between BM25 scores, vector similarity scores, and graph traversal scores.

The multi-signal confirmation effect is particularly valuable for legal retrieval: a provision appearing in top rankings of multiple independent signals is highly likely to be the correct provision. RRF surfaces that agreement without requiring calibration of signal weights.

4.5 BGE Cross-Encoder Reranking: Pair-Level Judgment

A cross-encoder reranker scores the query-provision pair jointly rather than independently. Where vector and BM25 score each chunk in isolation, the cross-encoder sees the query and chunk together and produces a relevance judgment on that specific pair.

This catches provisions in the right semantic neighborhood but not actually applicable to the query — a common failure mode in dense regulatory corpora where many provisions are topically adjacent but substantively different. The architecture limits cross-encoder computation to the top-k RRF candidates, capturing precision benefits at manageable latency cost. Diversity-aware reranking approaches such as MMR may be preferable when result coverage is a concern alongside precision.

5. Knowledge Governance Requirements Beyond Retrieval

Regardless of which retrieval architecture is chosen, certain knowledge governance requirements apply to any AI system operating on temporal legal corpora. These are not retrieval problems — they are data architecture prerequisites that should be resolved before retrieval design begins.

5.1 Document Version Lifecycle

Every document in the corpus should carry version identity: version identifier, effective date, supersession date (null if current), superseded-by reference, and current status flag. The ingestion pipeline should maintain this lifecycle — when a new version is ingested, the prior version is marked superseded rather than overwritten or deleted.

Without version lifecycle management, the corpus becomes an unordered mix of current and superseded provisions with no structural basis for distinction. This is the foundational prerequisite. Retrieval architecture improvements of any kind are secondary to it.

5.2 Ingestion as Knowledge Curation

For temporal legal corpora, document ingestion is recommended as a knowledge curation process rather than a purely technical pipeline. The curation step should establish: What is this document? What version? What does it supersede? What jurisdiction and product scope does it apply to? What structural relationships does it have with existing corpus documents?

These questions are difficult to answer reliably from document text alone without domain knowledge. Resolving them at ingestion time and encoding them in the data model is far more reliable than delegating resolution to a retrieval system at query time.

5.3 Immutable Decision Provenance

When an AI system generates a proposal or recommendation against a legal corpus, it is recommended that the system create and preserve an immutable record of: the query, the specific document chunks retrieved with their version identifiers and effective dates, the retrieval scores at generation time, the document versions that were current at generation time, the pipeline configuration in effect, and any confidence flags raised during retrieval.

This record is distinct from operational telemetry and from the generated output. It is the evidentiary basis for defending the output — or for demonstrating that a superseded document was in use when a decision was made and should be reconsidered under current rules. Once created, it should be immutable.

5.4 Explicit Uncertainty Expression

The system should be capable of expressing 'insufficient evidence' rather than always returning a best-available answer. When retrieval confidence signals fall below defined thresholds — no signal produces results above minimum relevance, the candidate pool contains only version-conflicted provisions, or the corpus contains no applicable provisions for the queried jurisdiction — a calibrated expression of uncertainty is the appropriate output.

6. Conclusion

Vector search alone is insufficient for mission-critical legal document retrieval. The failure mode — silent, confident retrieval of superseded, misscoped, or structurally inapplicable content — is a structural property of the approach, not an implementation deficiency that can be tuned away.

This paper recommends a hybrid retrieval architecture — BM25 lexical search, semantic vector retrieval with HyDE query expansion, knowledge graph traversal, and BGE cross-encoder reranking, fused by Reciprocal Rank Fusion — as one approach the author has found effective in practice. It is presented as a concrete example of the direction retrieval systems need to move, not as a uniquely correct solution. The field offers numerous alternative techniques (catalogued in Appendix C), and the appropriate architecture depends on corpus characteristics, operational constraints, and the specific failure modes most material to a given application.

What is not optional, regardless of retrieval approach, is knowledge governance. Document version lifecycle management, ingestion-time curation, and immutable decision provenance are prerequisites for any AI system operating on temporal legal corpora. A sophisticated retrieval pipeline operating on an uncurated, unversioned corpus will return the wrong answer with high

confidence. Addressing the retrieval architecture without addressing the data is solving the wrong problem.

The path to reliable AI-assisted legal document work runs through both: a corpus that encodes the structural and temporal properties of legal knowledge, and a retrieval architecture designed to operate on those properties.

Appendix A: Hybrid Retrieval — Signal Summary

Signal	Mechanism	Failure Mode Addressed
BM25 (Okapi)	Lexical — exact token match, TF-IDF with saturation + length norm	Precision vocabulary, exact term matching
Vector Search + HyDE	Semantic — approximate nearest-neighbor in embedding space; query expanded via hypothetical passage	Conceptual queries, vocabulary mismatch between query and document
Graph Traversal	Structural — entity relationship traversal, 1-hop and 2-hop, theme connections	Structural authority, jurisdictional relationships, version supersession (extended)
RRF Fusion	Rank fusion — ordinal position across all signals, k=60 constant	Signal space incompatibility, multi-signal confirmation
BGE Cross-Encoder	Pair-level judgment — transformer scores (query, passage) jointly	Topical adjacency without applicability, false semantic proximity

Appendix B: Glossary

- **BM25 (Okapi BM25).** A probabilistic lexical retrieval function scoring documents based on query term frequency and inverse document frequency, with term saturation and length normalization corrections.
- **Cross-encoder reranking.** A retrieval refinement approach where a transformer model scores query-document pairs jointly, producing more accurate relevance judgments than independent embedding-based scoring at higher computational cost.
- **HyDE (Hypothetical Document Embeddings).** A query expansion technique that generates a hypothetical document passage answering the query, then uses that passage's embedding for vector retrieval, improving recall for abstract queries by bridging query-space and document-space distributions.

- **Reciprocal Rank Fusion (RRF).** A rank fusion algorithm combining multiple ranked retrieval lists without requiring score normalization, using ordinal position rather than score magnitude.
- **Vector similarity search.** Approximate nearest-neighbor search in a dense embedding space, returning documents whose embeddings are closest to the query embedding by cosine or dot-product distance.
- **Version provenance.** An immutable record associating an AI-generated output with the specific document versions, effective dates, and retrieval signals that produced it, preserved at generation time.
- **Knowledge curation.** The process of establishing document identity, version lineage, jurisdictional scope, and structural relationships at ingestion time — as distinct from technical parsing and chunking.

Appendix C: Alternative and Complementary Retrieval Approaches

The hybrid architecture described in this paper represents one effective combination of retrieval techniques. The field offers a broad range of alternative and complementary approaches, each addressing the limitations of vector-only search from a different angle. Practitioners should evaluate these options against their specific corpus characteristics, latency and infrastructure constraints, and the failure modes most material to their application.

Structural and Chunking Strategies

- **Parent–child (hierarchical) chunking / 'parent document' retrieval.** Chunks are embedded at fine granularity for retrieval precision, but the parent section or document is returned as context. Preserves surrounding structure that flat chunking discards.
- **Heading/title field boosting (BM25F-style fielded retrieval).** Assigns higher weight to matches in title or heading fields than in body text. Particularly effective for regulatory documents where section titles carry strong topical signal.
- **Section-routing via table of contents.** Retrieves sections first using a ToC or heading index, then searches within the matched section. Preserves document hierarchy by routing before chunk-level retrieval.
- **Breadcrumb context embedding.** Attaches the full heading path (e.g., Document > Section 3 > Subsection 3.2) to every chunk at both lexical and embedding levels. Enables structural context to participate in retrieval without a separate graph layer.
- **Hierarchical summary index (RAPTOR-style).** Builds a tree of summaries from document leaves upward. Retrieval traverses from high-level summaries down to relevant leaf chunks, enabling both broad and precise retrieval in a single pass.

Query Expansion and Reformulation

- **Query rewriting + rank fusion (RAG-Fusion / RRF).** Generates multiple reformulations of the original query, retrieves independently for each, and fuses results via RRF. Improves recall by covering different phrasings of the same information need.
- **HyDE (Hypothetical Document Embeddings).** Generates a hypothetical answer passage and embeds it instead of the raw query, bridging the distributional gap between short queries and long document passages. Included in the recommended architecture; listed here for completeness.

Advanced Retrieval Models

- **Neural sparse retrieval (SPLADE / learned sparse expansion).** Learns sparse token-weight representations using a transformer, combining the interpretability of lexical retrieval with learned term expansion. Can outperform BM25 on out-of-vocabulary terms while remaining inverted-index compatible.
- **Late-interaction retrieval (ColBERTv2 / multi-vector).** Encodes queries and documents as sets of token-level vectors; scores via MaxSim late interaction at query time. Achieves cross-encoder-level precision at closer to bi-encoder latency, at the cost of significantly higher index storage.

Reranking and Result Refinement

- **Cross-encoder reranking (BGE reranker class).** Scores query-document pairs jointly using a transformer, producing high-precision relevance judgments. Included in the recommended architecture; listed here for completeness.
- **Diversity-aware reranking (MMR / coverage constraints).** Maximal Marginal Relevance and similar approaches balance relevance with result diversity, reducing redundancy in the returned set. Useful when comprehensive coverage across provisions matters as much as top-1 precision.

Graph and Structural Retrieval

- **Graph-guided retrieval expansion (GraphRAG local→global).** Uses a knowledge graph to expand retrieval from local entity-level search to global community-level synthesis. Particularly effective for queries that require reasoning across multiple related documents rather than finding a single provision.

Retrieval Quality Control

- **Retrieval-quality gating / corrective loops (Self-RAG / CRAG).** Evaluates retrieval quality before generation and triggers corrective retrieval if the initial results are insufficient. Self-RAG generates retrieval and relevance tokens inline; CRAG uses a lightweight evaluator to gate or augment retrieval results.
- **Contextual compression / span extraction.** Trims retrieved chunks to the specific spans most relevant to the query before passing them to the LLM. Reduces context window

consumption and noise from partially relevant chunks, improving generation quality for long-document retrieval.

Note: This list is illustrative, not exhaustive. The retrieval research literature evolves rapidly and new approaches continue to emerge. The appropriate selection depends on corpus size and structure, query patterns, latency constraints, infrastructure, and the specific failure modes most consequential to the application.